COSC 42: Distributed Computing

Problem 1. ***n*-Process Locks from Two-Process Locks**.
Reduce n-process locks to two process locks. That is, let $n \geq 2$ be any fixed and known integer, and let L_2 be a solution to the mutual exclusion problem for two-processes, construct by reduction a lock L_n that solves the mutual exclusion problem for n-processes. Your reduction should only use the read and write shared-memory instructions (i.e., it should not use RMW instruction) and it should satisfy the following requirements:

(LLF) If L_2 satisfies Mutex and Live-Lock Freedom but not Starvation Freedom for two processes, then L_n should satisfy Mutex and Live-Lock Freedom but not Starvation Freedom for n processes.

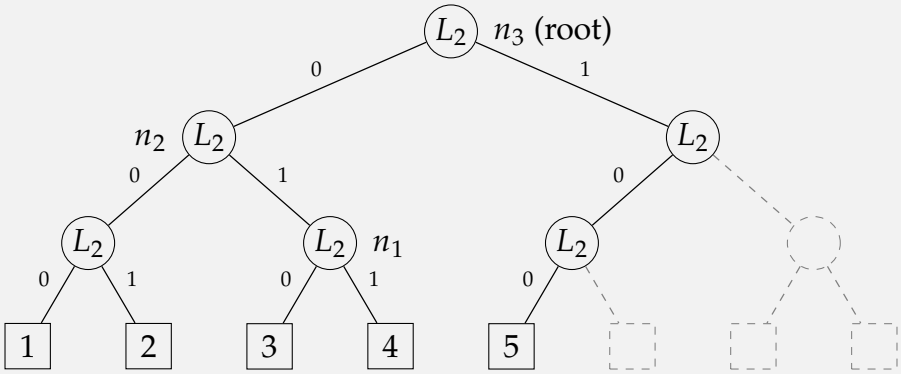
(SF) If L_2 satisfies Mutex and Starvation Freedom for two processes, then L_n should satisfy Mutex and Starvation Freedom for n processes.

You must design the reduction and prove that it satisfies the requirements (LLF) and (SF). You may assume that n is fixed and known and that the processes are named $1, 2, \ldots, n$.

Solution.

We can construct a lock L_n that solves the mutual exclusion problem for n-processes by constructing a tournament bracket for the n processes.

We construct a complete binary tree of height $h = \lceil \log_2 n \rceil$ with the first n leaf nodes corresponding to processes $1, \ldots, n$ (the remaining leaves are unused). Each internal node acts as an independent L_2 lock, and its two roles 0 and 1 correspond to its left and right subtrees. Where if they 'win' that node's L_2 lock, they move to the next node instead of entering CS. Also note that by 'holds' we mean that a process has finished a node's Try section but hasn't yet started its Exit section.



The tournament tree for $n = 5$, so $h = \lceil \log_2 5 \rceil = 3$. Each circle is an independent L_2 lock, and the edge labels give the side (role) a process uses at that lock. Dashed nodes are unused.

Let $n_1, \ldots, n_h$ be the path from a process p's leaf nodes parent to the root node, and let $r_i(p) \in \{0, 1\}$ indicate which side of node n_i contains p. Then for a process p we define it's algorithm for the Try and Exit sections of the L_n lock to be:

```
Try_n(p):
    for i in range(1, h+1):
        Try_{n_i}(r_i(p))

    enter CS

Exit_n(p):
    for i in range(h, 0, -1):
        Exit_{n_i}(r_i(p))
    return to Remainder
```

Note that a process holds every 'win' it acquires until it leaves the Critical Sectionm and that it releases its wins in reverse order, starting from the root and working back down to its leaf. We say a process holds a lock on a node if it has finished that node's Try section but has not yet finished its Exit Section. Every process in the CS of L_n holds the root.

Mutex

We will show via induction on the height of the node that Mutex is satisfied.

Base Case (height $= 1$): For parents of leaves, each side only has one process, and hence by the Mutex of the L_2 of that node this is clear.

Inductive Hypothesis (height $= h - 1$): Suppose that for a higher node v, at most one process approaches it from each side (by Mutex of its child c).

Inductive Step: Let v be a node of height h, and let c be one of its child nodes. Any process on that side of v already has a lock on c. Then, since the locks are released in reverse order (root to leaves), the process doesn't start Exit_c until it has finished Exit_v, hence the process

holds c the whole time it is in v. By the inductive hypothesis, c's L_2 lock is used as a valid two-process lock, so by Mutex of c's L_2 at most one process can hold c. Thus at most one process can be using that side of v.

Hence the root's L_2 lock is also valid and by its Mutex at most one process can hold the root, and thus L_n satisfies Mutex.

(LLF)

Suppose L_2 satisfies Live-Lock Freedom, so if some process is in an L_2 Try section and nobody holds that lock, then eventually some process wins it. Assume for contradiction that some process stays in Try_n forever and that after some point no process ever enters the CS again.

Then once all previously started CS and Exit sections finish, no process ever releases a node again, since processes only release nodes after leaving the CS. Thus processes only move up the tree and each process can move up at most h times. So after some point no one wins any more locks. Thus every process is either in the Remainder section or stuck forever in the Try section of some node.

Let v be the highest node where a process q is stuck in the Try section. Since no one wins the lock for v again, by the Live Lock Freedom of v's L_2 someone must be holding v forever. Let that process be s. If v is the root then s eventually enters and releases the CS, a contradiction. Otherwise, s holds v but never releases it, so s is not in the CS and must be stuck at some node above v. This contradicts our choice of v as the highest node where a process is stuck in the Try section.

Thus some process eventually enters the CS, so L_n satisfies Live-Lock Freedom. To show that it is not starvation-free when L_2 is not, let only two processes try, one from each side of the root. They can get their lower locks since nobody else is trying for those locks. At the root, let them take steps in the same order as an L_2 execution where one process starves. The extra steps for the lower locks all finish, so the execution is still fair. Thus same process waits forever at the root and never enters the CS, and L_n is not starvation-free.

(SF)

Suppose L_2 satisfies Starvation Freedom, so any process in an L_2 Try section eventually wins as long as whoever holds that lock eventually releases it.

Let p be any process in Try_n with path $n_1, \ldots, n_h$. We will show that if p is in the Try section of n_i, then p eventually wins n_i, and eventually enters the CS via induction on i that going down from the root ($i = h$) to $i = 1$.

Base Case ($i = root$): At the root, whatever process is using the CS eventually is released. Note that Exit_n releases the root first. Then by the starvation freedom of the L_2 of the root p eventually wins it and enters the CS.

Inductive Hypothesis: Suppose that the claim holds for every node n_j for $j > i$.

Inductive Step: Let q be a process holding n_i. Since q is above n_i, it is in one of three places:

1. in the Try section of some n_j with $j > i$. Then by IH q eventually enters the CS.
2. in the CS, by starvation freedom of the L_2 of the root q eventually leaves it.
3. in Exit_n.

In every case q eventually reaches Exit_n, which releases its locks from the root down to n_i, so whoever holds n_i always releases it, and by Starvation Freedom of n_i's L_2 lock, p eventually wins n_i. Then p moves on to the Try section of n_{i+1}, and by the inductive hypothesis p eventually enters the CS.

Thus any process that starts Try_n eventually wins every node on its path and enters the CS. Hence L_n satisfies Starvation Freedom.